Polymorphism

Definition
Polymorphism means "many forms". It refers to the ability of an entity (like a function or object) to perform different actions based on the context.

How Polymorphism works?
Technically, Polymorphism allows the same method, function, or operator to behave differently depending on the object it is working with. This makes code more flexible and reusable.

Advantage of Polymorphism

It makes code more flexible and reusable.

Example: In a backend payment system, multiple payment options are available, such as Credit Card, UPI, NetBanking, and Wallet. All payment types use a common method named processPayment(), but different implementations:
• NetBanking Payment: redirects to bank login

The method name remains the same, but the action changes based on the payment type.

Types of Polymorphism:

Polymorphism refers to the ability of the same method or operation to behave differently based on object or context. It mainly includes compile-time and runtime polymorphism (as shown in Figure 3.6)

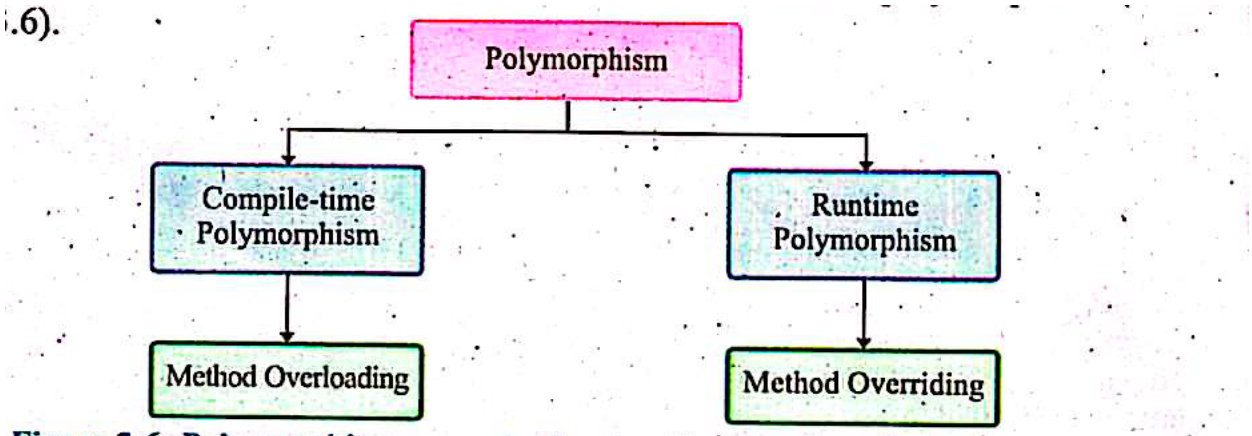


Figure 3.6: Polymorphism example showing the same function behaving differently

1. Compile-time Polymorphism (Method Overloading): Compile-time polymorphism means deciding which method or operation to run during compilation, usually through method or operator overloading. Languages like Java or C++ support this. But python does not support true method overloading based on function signatures. like statically typed languages (e.g., Java or C++). Because Python is dynamically

typed, a single function can accept arguments of different types, making signature-based overloading unnecessary. Instead, similar behavior can be achieved using default arguments, variable-length arguments (*args, **kwargs), or conditional logic inside the function.

Example: This code demonstrates method overloading using default and variable-length arguments. The multiply() method works with different numbers of inputs, mimicking compile-time polymorphism.

```
class Calculator:
    def multiply(self, a=1, b=1, *args):
        result = a * b
        for num in args:
            result *= num
        return result

calc = Calculator()
print(calc.multiply())          # 1    (defaults)
print(calc.multiply(4))         # 4
print(calc.multiply(2, 3))      # 6
print(calc.multiply(2, 3, 4))   # 24
```

Short Questions

1. What is polymorphism?
2. What does the word “polymorphism” mean?
3. Does Python support true Method Overloading? Give reason.
4. How Polymorphism works?
5. Name the types of polymorphism.
6. What is compile-time polymorphism?

Long Questions

1. What is polymorphism in OOP? Explain its types with examples.
2. How does polymorphism make code more flexible and reusable? Explain with example.
3. Write a Python program to demonstrate compile time polymorphism using default arguments, variable-length arguments.